

Catering Operations Management System — Database Schema

Stack: PostgreSQL. Conventions: `id` = `BIGSERIAL PRIMARY KEY`, all money = `NUMERIC(12, 2)` (never float — rounding errors on invoices are how you lose trust with clients), all dates stored as `DATE`/`TIMESTAMP` in ISO format, formatted DD/MM/YYYY only at the frontend layer. VAT = fixed 16% constant, not a per-row field, unless you plan to support VAT-exempt clients later.

MODULE 1: Bookings

Client

Column	Type	Notes
id	BIGSERIAL PK	
full_name	VARCHAR(150)	
phone	VARCHAR(20)	Kenyan format, e.g. 2547XXXXXXXX
email	VARCHAR(150)	nullable
company_name	VARCHAR(150)	nullable — for corporate clients
address	TEXT	nullable
created_at	TIMESTAMP	

EventType (lookup table, not enum — you'll want to add types without a redeploy)

Column	Type	Notes
id	SERIAL PK	
name	VARCHAR(50)	wedding, corporate, funeral, birthday, conference

Booking

Column	Type	Notes
id	BIGSERIAL PK	
client_id	BIGINT FK → Client	
event_type_id	INT FK → EventType	
event_date	DATE	
start_time	TIME	
end_time	TIME	
venue_location	VARCHAR(255)	
guest_count	INT	
menu_package_id	BIGINT FK → MenuPackage	nullable until client picks
status	VARCHAR(20)	pending / confirmed / cancelled / completed
notes	TEXT	
created_by	BIGINT FK → AppUser	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Conflict check (double-booking staff/venue on same date) is **application logic** at insert time — a query against (event_date) + (venue_location) overlap, not its own table.

MODULE 2: Menu Costing

Analogy: (MenuItem) is a single ingredient's cost card, (MenuPackage) is the printed menu clients pick from, (MenuPackageItem) is the join telling you which ingredients make up which package.

MenuItem

Column	Type	Notes
id	BIGSERIAL PK	
name	VARCHAR(150)	e.g. "Pilau", "Beef stew"
category	VARCHAR(30)	starter / main / dessert / beverage
unit_cost	NUMERIC(12,2)	cost to produce per portion
unit	VARCHAR(20)	kg, litre, portion

MenuPackage

Column	Type	Notes
id	BIGSERIAL PK	
name	VARCHAR(150)	e.g. "Silver Buffet Package"
service_type	VARCHAR(30)	buffet / plated / self-service
price_per_head	NUMERIC(12,2)	client-facing price
min_guests	INT	

MenuPackageItem (join table)

Column	Type	Notes
id	BIGSERIAL PK	
menu_package_id	BIGINT FK → MenuPackage	
menu_item_id	BIGINT FK → MenuItem	
portion_size	NUMERIC(8,2)	per guest

CostCalculation (snapshot per booking — prices change over time, don't recompute historic bookings off live MenuItem costs)

Column	Type	Notes
id	BIGSERIAL PK	
booking_id	BIGINT FK → Booking	
total_ingredient_cost	NUMERIC(12,2)	
total_service_cost	NUMERIC(12,2)	staff + equipment
total_cost	NUMERIC(12,2)	
generated_at	TIMESTAMP	

MODULE 3: Quotation & Invoice

Analogy: Quotation is the proposed price, Invoice is the actual bill once they say yes, Payment is proof cash actually moved.

Quotation

Column	Type	Notes
id	BIGSERIAL PK	
booking_id	BIGINT FK → Booking	
quotation_number	VARCHAR(30)	unique, e.g. QT-2026-0042
subtotal	NUMERIC(12,2)	
vat_amount	NUMERIC(12,2)	16% of subtotal
total	NUMERIC(12,2)	
status	VARCHAR(20)	draft / sent / approved / rejected
valid_until	DATE	
created_at	TIMESTAMP	

QuotationLineItem

Column	Type	Notes
id	BIGSERIAL PK	
quotation_id	BIGINT FK → Quotation	
description	VARCHAR(255)	
quantity	INT	
unit_price	NUMERIC(12,2)	
line_total	NUMERIC(12,2)	

Invoice

Column	Type	Notes
id	BIGSERIAL PK	
quotation_id	BIGINT FK → Quotation	
booking_id	BIGINT FK → Booking	
invoice_number	VARCHAR(30)	unique
amount_due	NUMERIC(12,2)	
amount_paid	NUMERIC(12,2)	default 0
balance	NUMERIC(12,2)	computed: amount_due - amount_paid
due_date	DATE	
status	VARCHAR(20)	unpaid / partial / paid / overdue
created_at	TIMESTAMP	

Payment

Column	Type	Notes
id	BIGSERIAL PK	
invoice_id	BIGINT FK → Invoice	
amount	NUMERIC(12,2)	
payment_method	VARCHAR(20)	mpesa / cash / bank
mpesa_reference	VARCHAR(30)	nullable, required if method = mpesa
payment_date	TIMESTAMP	
recorded_by	BIGINT FK → AppUser	

MODULE 4: Staff & Inventory Management

Staff

Column	Type	Notes
id	BIGSERIAL PK	
full_name	VARCHAR(150)	
phone	VARCHAR(20)	
role	VARCHAR(30)	chef / waiter / driver / supervisor
hourly_rate	NUMERIC(10,2)	nullable
status	VARCHAR(20)	active / inactive

StaffAssignment (join: who works which event)

Column	Type	Notes
id	BIGSERIAL PK	
booking_id	BIGINT FK → Booking	
staff_id	BIGINT FK → Staff	
role_for_event	VARCHAR(50)	may differ from default role
shift_start	TIMESTAMP	
shift_end	TIMESTAMP	
status	VARCHAR(20)	assigned / confirmed / no-show / completed

InventoryItem

Column	Type	Notes
id	BIGSERIAL PK	
name	VARCHAR(150)	e.g. "Chafing dish", "Plates"
category	VARCHAR(30)	equipment / consumable
unit	VARCHAR(20)	piece, kg, box
quantity_in_stock	INT	
reorder_level	INT	triggers low-stock flag
unit_cost	NUMERIC(12,2)	

InventoryAllocation (equipment assigned to a specific event — this is what "flags resource shortfalls" from your proposal)

Column	Type	Notes
id	BIGSERIAL PK	
booking_id	BIGINT FK → Booking	
inventory_item_id	BIGINT FK → InventoryItem	
quantity_allocated	INT	
quantity_returned	INT	nullable until event ends
condition_notes	TEXT	damage/loss notes

StockTransaction (audit trail of stock movement — restocks, allocations, returns)

Column	Type	Notes
id	BIGSERIAL PK	
inventory_item_id	BIGINT FK → InventoryItem	
transaction_type	VARCHAR(20)	restock / allocation / return / damage
quantity	INT	
reference_booking_id	BIGINT FK → Booking	nullable
transaction_date	TIMESTAMP	

MODULE 5: Reports & Admin

AppUser (system login, referenced by other modules above)

Column	Type	Notes
id	BIGSERIAL PK	
full_name	VARCHAR(150)	
email	VARCHAR(150)	unique
password_hash	VARCHAR(255)	
role	VARCHAR(20)	admin / manager / staff
last_login	TIMESTAMP	nullable
created_at	TIMESTAMP	

AuditLog

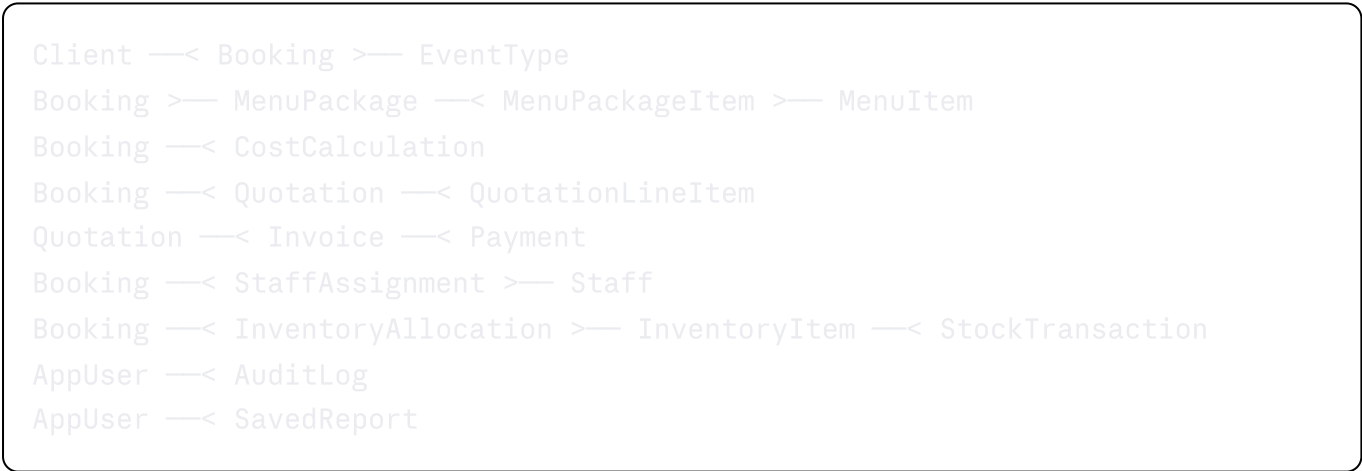
Column	Type	Notes
id	BIGSERIAL PK	
user_id	BIGINT FK → AppUser	
action	VARCHAR(100)	e.g. "APPROVED_QUOTATION"
entity_type	VARCHAR(50)	Booking, Invoice, etc.
entity_id	BIGINT	
timestamp	TIMESTAMP	

SavedReport (optional — only needed if you cache/export reports rather than always computing live)

Column	Type	Notes
id	BIGSERIAL PK	
report_type	VARCHAR(50)	financial / operational / inventory
date_range_start	DATE	
date_range_end	DATE	
generated_by	BIGINT FK → AppUser	
generated_at	TIMESTAMP	
file_path	VARCHAR(255)	if exported to PDF/Excel

Most reports (revenue by month, top menu items, staff utilization) should be **views/queries**, not stored tables — only persist a `SavedReport` row if you need an export history.

Relationship Summary (who points to whom)



Things to decide before you build this, not after

- Booking status vs Invoice status vs Payment status** are three separate state machines — don't collapse them into one "status" field, you'll paint yourself into a corner.
- Offline caching:** since you need offline support, `Booking`, `InventoryAllocation`, and `Payment` are the tables most likely to be created/edited offline — give them a client-generated UUID alongside the serial `id` for conflict-free sync.
- Soft delete:** for `Client`, `Invoice`, `Payment` — add a `deleted_at` nullable column instead of hard deletes. You cannot legally lose financial records.